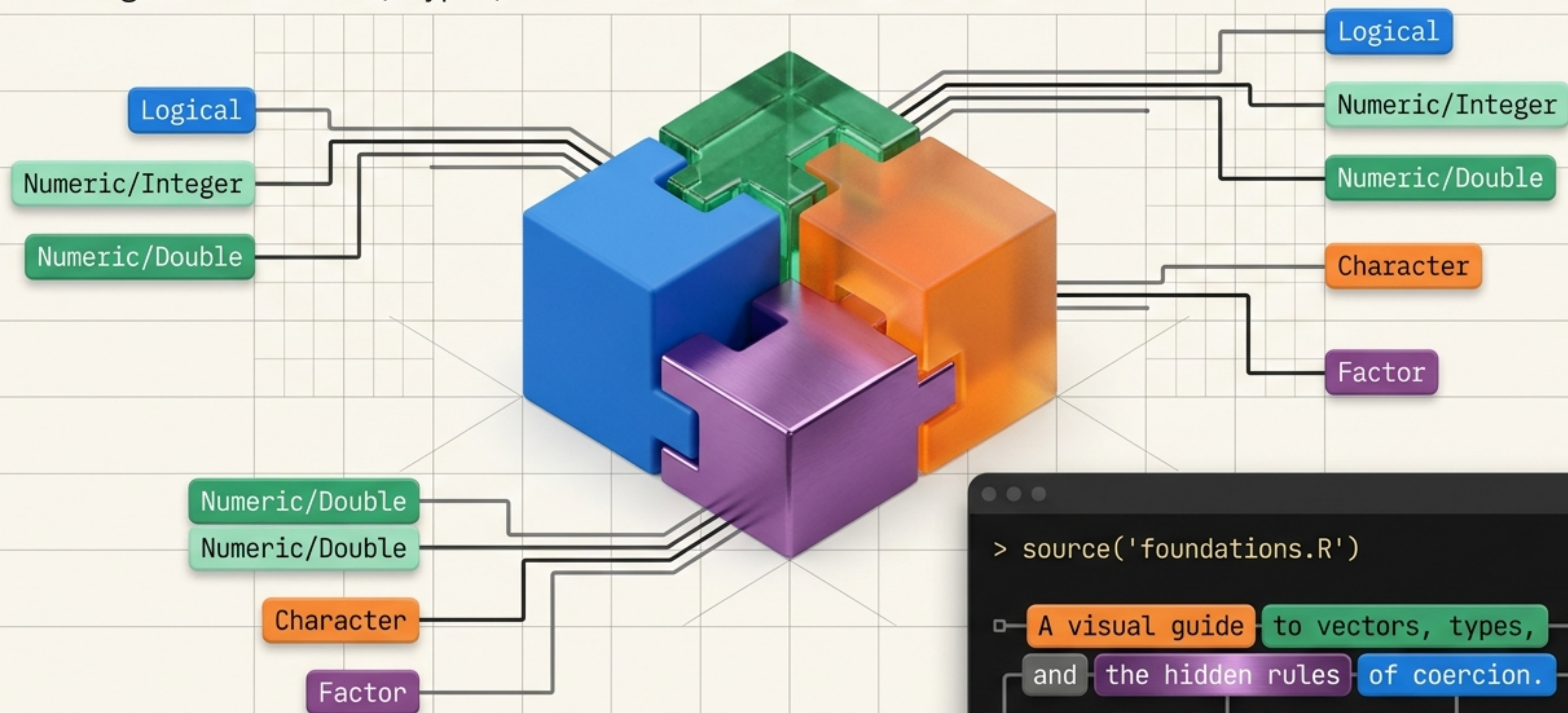


The Architecture of R Data

A visual guide to vectors, types, and the hidden rules of coercion.



```
> source('foundations.R')
```

```
▣ A visual guide to vectors, types,  
and the hidden rules of coercion.
```

Everything in R is an object.

To create an object, we assign a value to a variable. R uses the `<-` operator to bind data to a name (Keyboard shortcut: Alt + -). To view the object, simply call its name.

```
> x <- 15
```

```
> x
```

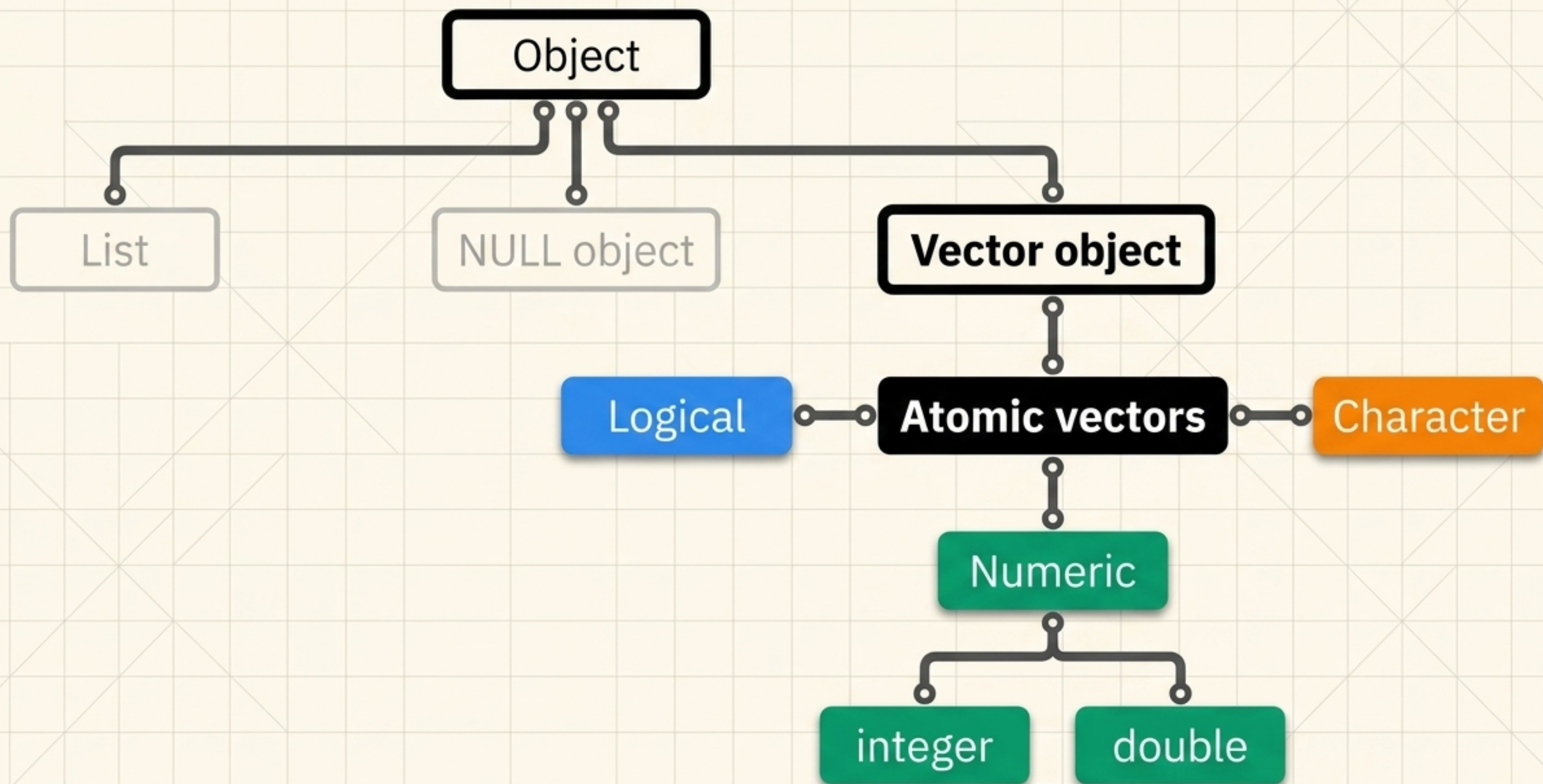
```
[1] 15
```

```
> a <- 'space'
```

```
> a
```

```
[1] 'space'
```

The Hierarchy of R Objects



The fundamental data structure is the Vector.

A vector is a contiguous container that stores elements. The golden rule of atomic vectors: Every element inside must be of the exact same data type.

```
> concat <- c(15, 20, 25, 30)
> concat
```

The Vector Container



← A numeric vector of length 4, bound together by the concatenate function `c()`.

The Four Primary Elements

Numeric

```
> numeric <- c(1, 2, 3, 4)
> str(numeric)
```

```
num [1:4] 1 2 3 4
```

Character

```
> character <- c('a', 'b', 'c', 'd')
> str(character)
```

```
chr [1:4] 'a' 'b' 'c' 'd'
```

Logical

```
> logical <- c(TRUE, FALSE, FALSE, TRUE)
> str(logical)
```

```
logi [1:4] TRUE FALSE FALSE TRUE
```

Factor

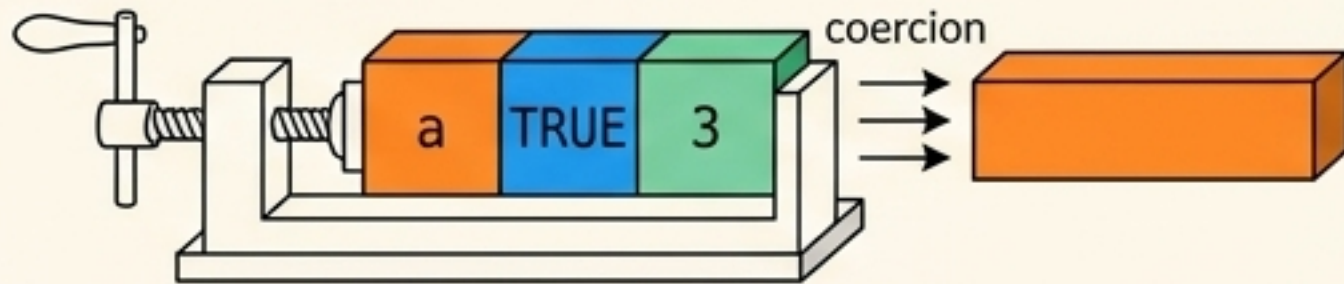
Used for categorical data

```
> factor <- factor(rep(c(0,1),2),
  levels=c(0,1), labels=c('D-', 'D+'), ordered=T)
> str(factor)
```

```
Ord.factor w/ 2 levels 'D-'<'D+': 1 2 1 2
```

Methods for building vectors from scratch.

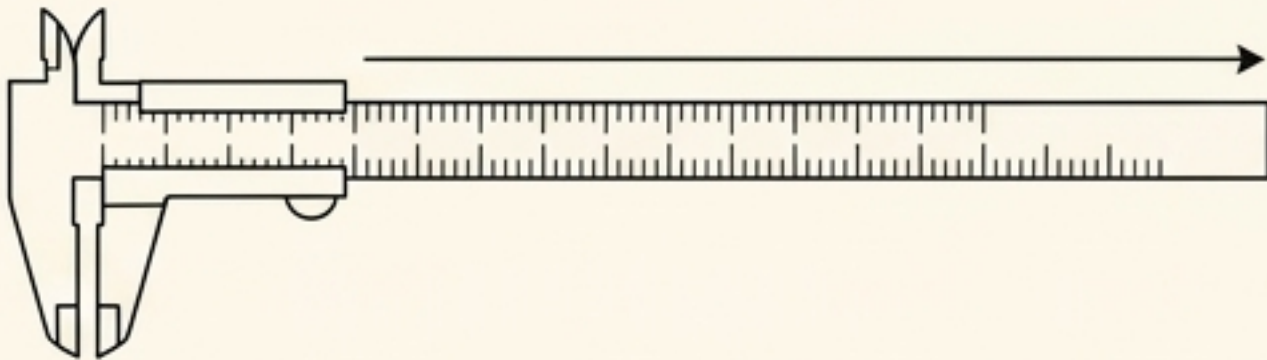
The Glue (Concatenate)



```
c('a', 'b', 'c', 1:3, TRUE)
```

```
chr [1:7] 'a' 'b' 'c' '1' '2' '3' 'TRUE'
```

The Ruler (Sequencing)



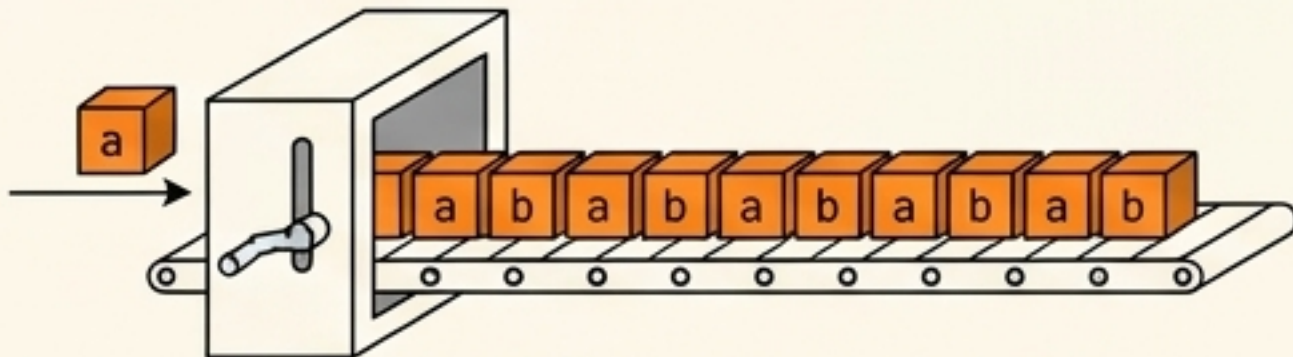
```
1:10
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(from = 0.5, to = 5, by = 0.5)
```

```
[1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

The Stamp (Repeating)



```
rep(c('a','b'), times=5)
```

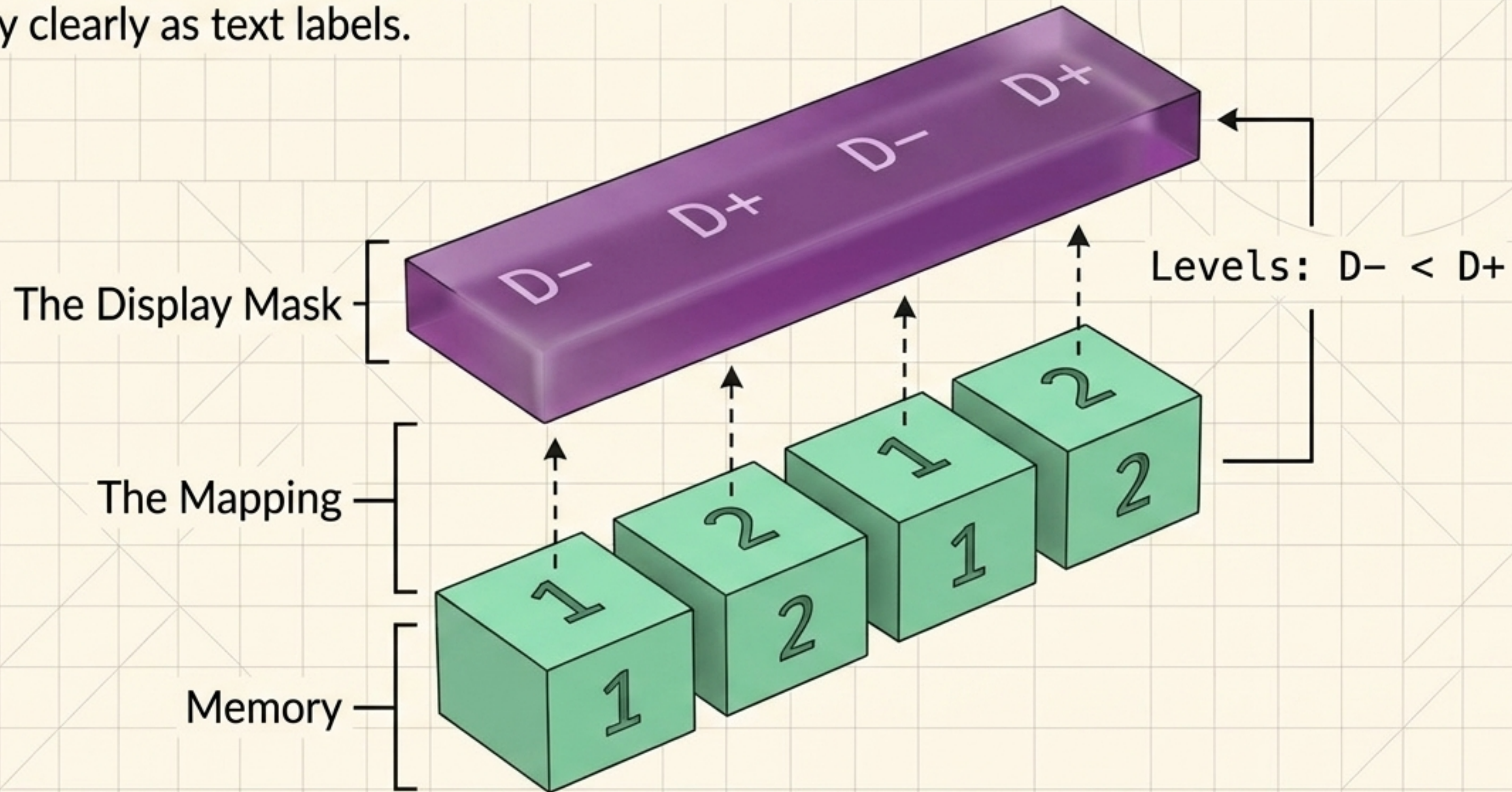
```
[1] 'a' 'b' 'a' 'b' 'a' 'b' 'a' 'b' 'a' 'b'
```

```
rep(1:2, each=5)
```

```
[1] 1 1 1 1 1 2 2 2 2 2
```

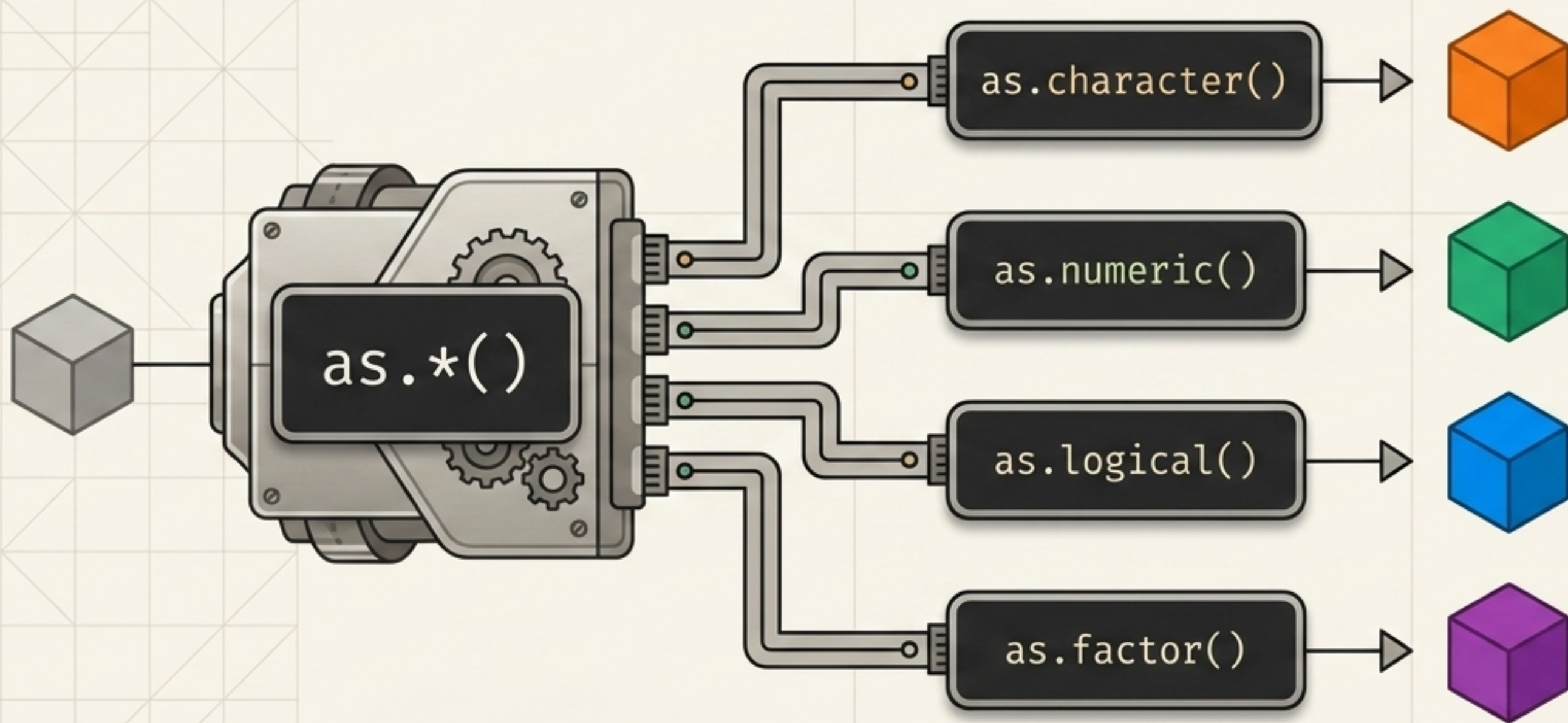
Factors: Integers wearing a mask.

We usually prefer factors to character strings for categorical data. They store **efficiently in memory as integers**, but display clearly as text labels.



Explicit Coercion: Forcing data to change states.

You can manually convert a vector from one data type to another using `as.*()` functions. The data physically changes its underlying representation to match the new type.



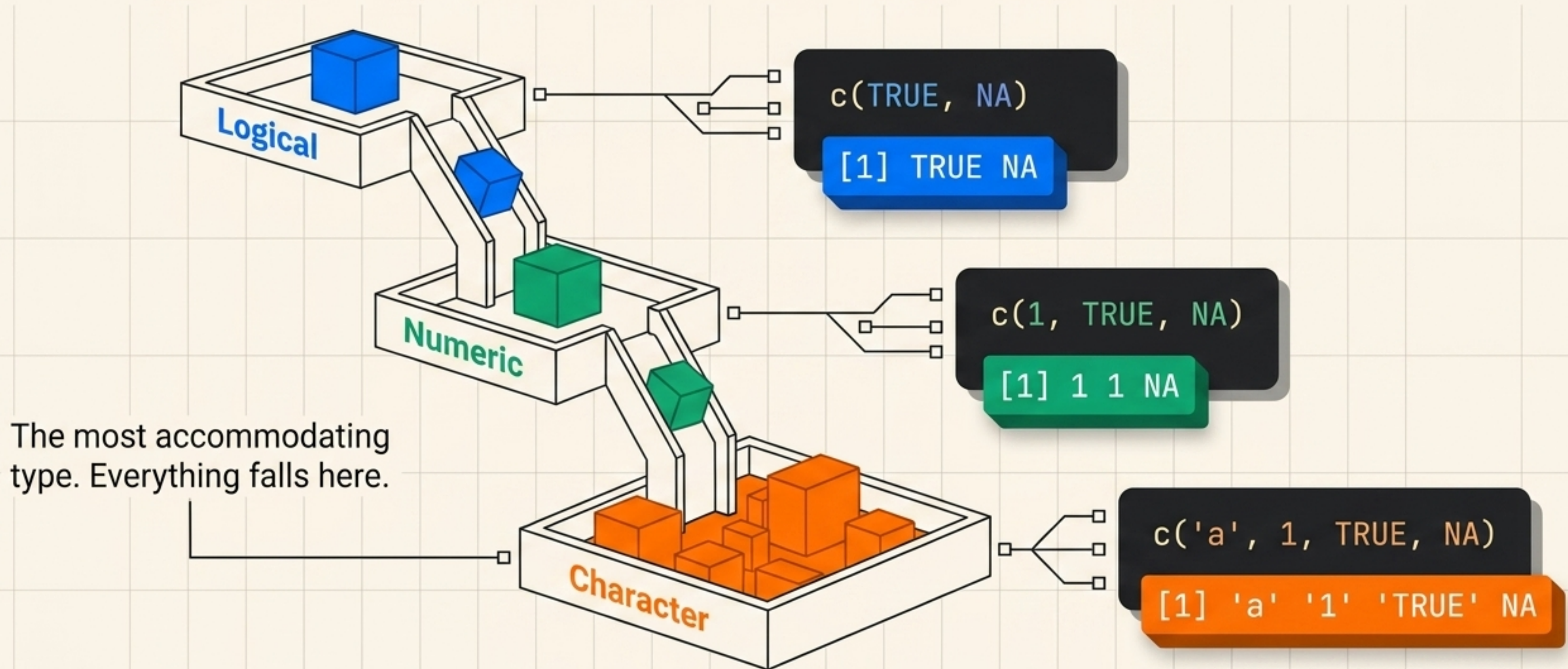
The Rules of Transformation

The Coercion Matrix

		Target Type	
		Target Type: Numeric	Target Type: Character
Source Type	Source Type: Logical	<pre>TRUE → 1, FALSE → 0</pre> 10	<pre>TRUE → 'TRUE' FALSE → 'FALSE'</pre> 'TRUE''FALSE'
	Source Type: Numeric	<pre>0 → FALSE Any other number (e.g., -1, 1.5) → TRUE</pre> FALSETRUE	<pre>1:10 → '1', '2', '3'...</pre> '1''2''3'...'10'
	Source Type: Character	<pre>'TRUE'/'T' (any case) → TRUE 'FALSE'/'F' → FALSE Other text → NA.</pre> TRUEFALSENA	<pre>Only strings with numbers ('0.5', '-1.5') convert. Letters ('a'), words ('TRUE'), and math expressions ('(3-1)') result in NA + Warning: NAs introduced by coercion</pre> 0.5-1.5NA⚠

Implicit Coercion & The Hierarchy of Power

When different data types are forced into a single vector container, R silently coerces them to the most accommodating type to maintain the one type per container rule.



The Two Faces of Numeric Data

Under the hood, numeric atomic vectors are split into two precise architectures. Knowing the difference is critical to avoiding invisible errors.

	Double	Integer
Default Status	R's default numeric type	Requires explicit syntax
Nature	Approximations	Exact values
Syntax Check	<pre>typeof(2) → 'double'</pre>	<pre>typeof(2L) → 'integer'</pre>
Equality Check	<pre>2 == 2L → TRUE</pre>	

The Illusion of Exactness

Because doubles are approximations, mathematical operations can leave behind microscopic decimal dust. Relying on **strict equality** (==) with **doubles will break your code**.

Phase 1
(The Anomaly)

```
> sqrt(2)^2 == 2  
[1] FALSE
```

Phase 2
(The Diagnosis)

```
> typeof(sqrt(2)^2)  
[1] 'double'
```

```
> sqrt(2)^2 - 2  
[1] 4.440892e-16
```

Phase 3
(The Solution)

```
> dplyr::near(sqrt(2)^2, 2)  
[1] TRUE
```

The Anatomy of Special Values

Doubles require a wider vocabulary than integers to handle the approximations and limits of floating-point math.

Integers

Have exactly one special value: NA (Missing).

```
> NA + 5
```

NA

```
> mean(rep(c(1, NA),  
each = 100), na.rm = F)
```

NA

Doubles

Have four special values:
NA, -Inf, NaN, Inf.

```
> c(-1, 0, 1) / 0
```

[1]

-Inf

NaN

Inf

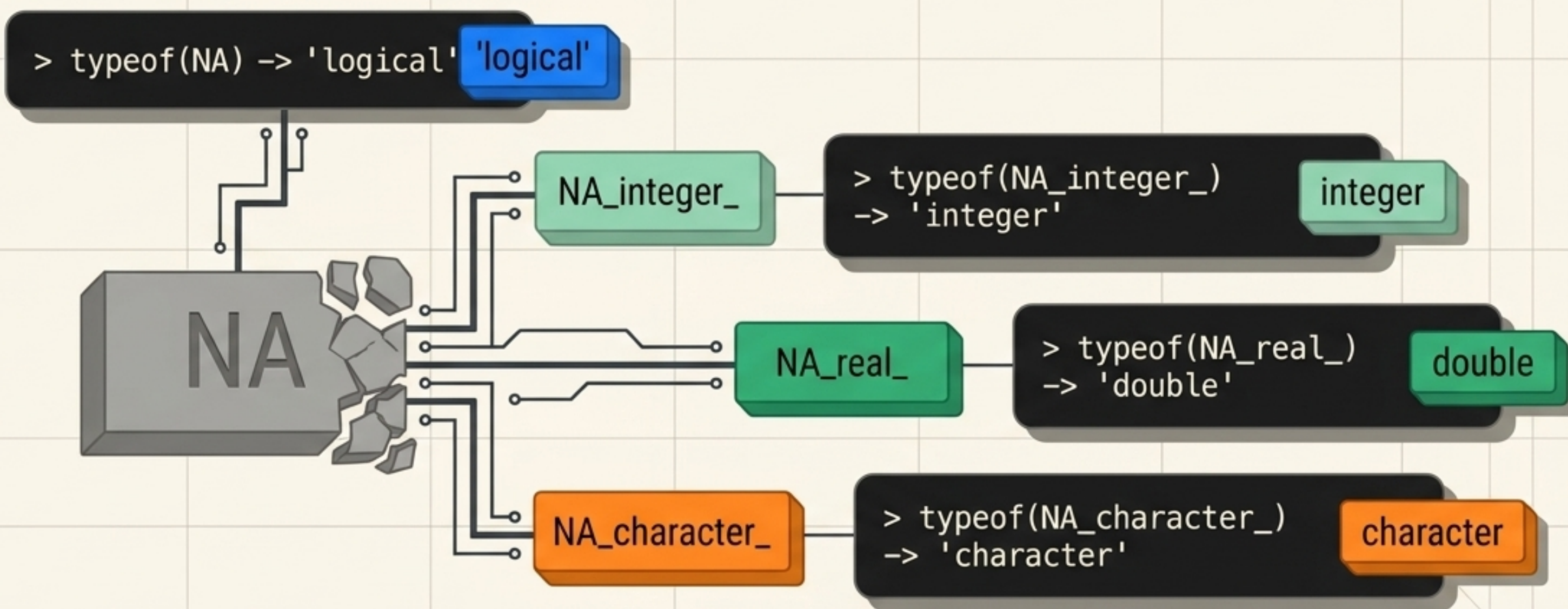
```
> -Inf + Inf
```

NaN

Not a Number

The Chameleon Nature of Missing Data

By default, NA is a logical data type. But remember the golden rule: a vector container can only hold one type of data. To exist inside numeric or character vectors without triggering mass coercion, NA has hidden, type-specific variants.



The architecture of R ensures that even emptiness conforms to the rules of the container.